



Chapter 11 – Reliability Engineering

Topics covered



- ✧ Availability and reliability
- ✧ Reliability requirements
- ✧ Fault-tolerant architectures
- ✧ Programming for reliability
- ✧ Reliability measurement

Software reliability



- ✧ In general, **software customers expect all software to be dependable.**
 - However, for **non-critical applications** - willing to accept some system failures.
- ✧ **critical systems** - have very high reliability requirements and special software engineering techniques may be used to achieve this.
 - Medical systems
 - Telecommunications and power systems
 - Aerospace systems

Faults, errors and failures



Term	Description
Human error or mistake	Human behavior that results in the introduction of faults into a system. For example, in the wilderness weather system, a programmer might decide that the way to compute the time for the next transmission is to add 1 hour to the current time. This works except when the transmission time is between 23.00 and midnight (midnight is 00.00 in the 24-hour clock).
System fault	A characteristic of a software system that can lead to a system error. The fault is the inclusion of the code to add 1 hour to the time of the last transmission, without a check if the time is greater than or equal to 23.00.
System error	An erroneous system state that can lead to system behavior that is unexpected by system users. The value of transmission time is set incorrectly (to 24.XX rather than 00.XX) when the faulty code is executed.
System failure	An event that occurs at some point in time when the system does not deliver a service as expected by its users. No weather data is transmitted because the time is invalid.

Faults and failures



- ✧ **Failures** are a usually a result of **system errors that are derived from faults in the system**
- ✧ However, **faults do not necessarily result in system errors**
 - The erroneous system state resulting from the fault may be transient and 'corrected' before an error arises.
 - The faulty code may never be executed.
- ✧ **Errors do not necessarily lead to system failures**
 - The error can be corrected by built-in error detection and recovery
 - The failure can be protected against by built-in protection facilities. These may, for example, protect system resources from system errors

Fault management



✧ Fault avoidance

- The system is developed in such a way that human error is avoided and thus system faults are minimised.
- The development process is organised so that faults in the system are detected and repaired before delivery to the customer.

✧ Fault detection

- Verification and validation techniques are used to discover and remove faults in a system before it is deployed.

✧ Fault tolerance

- The system is designed so that faults in the delivered software do not result in system failure.

Reliability achievement



✧ Fault avoidance

- Development techniques are used that either minimise the possibility of mistakes or trap mistakes before they result in the introduction of system faults.

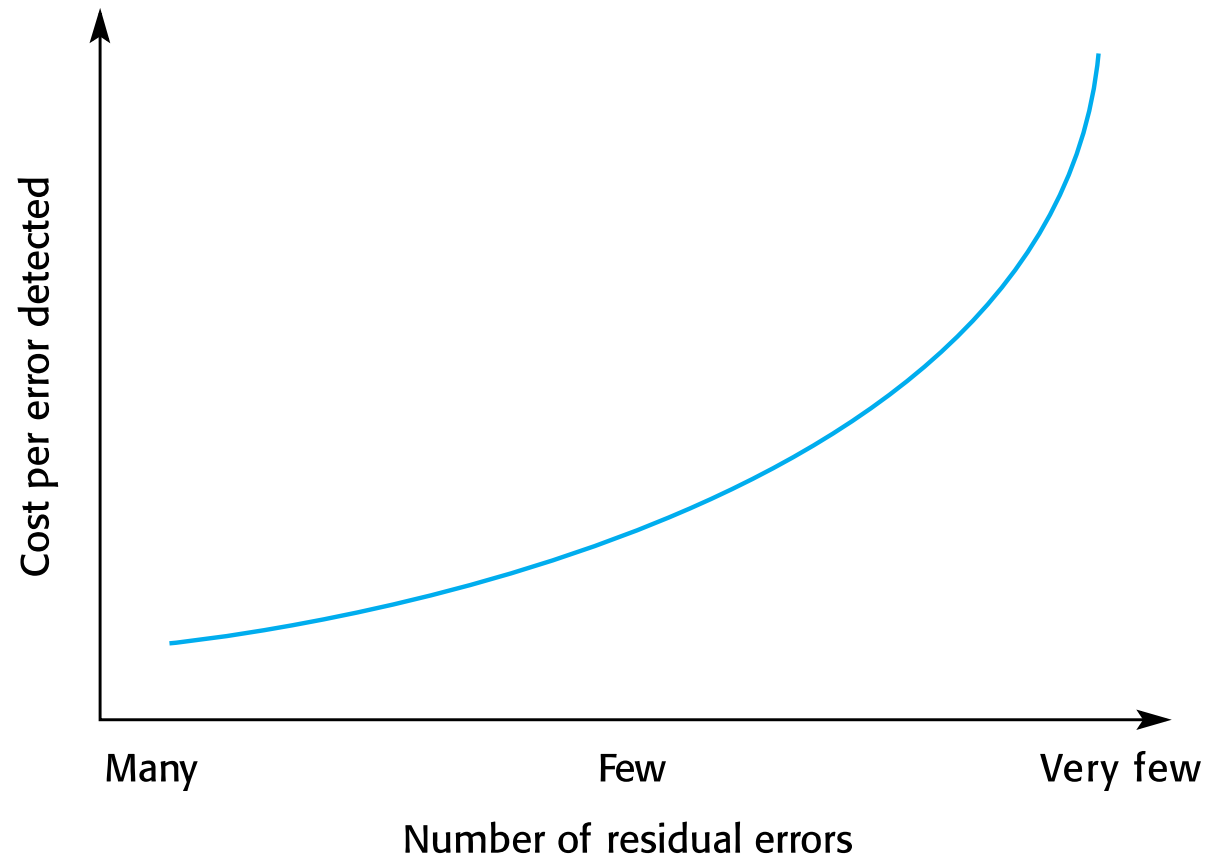
✧ Fault detection and removal

- Verification and validation techniques are used that increase the probability of detecting and correcting errors before the system goes into service are used.

✧ Fault tolerance

- Run-time techniques are used to ensure that system faults do not result in system errors and/or that system errors do not lead to system failures.

The increasing costs of residual fault removal



Availability and reliability

Availability and reliability



✧ Reliability

- The probability of failure-free system operation over a specified time in a given environment for a given purpose

✧ Availability

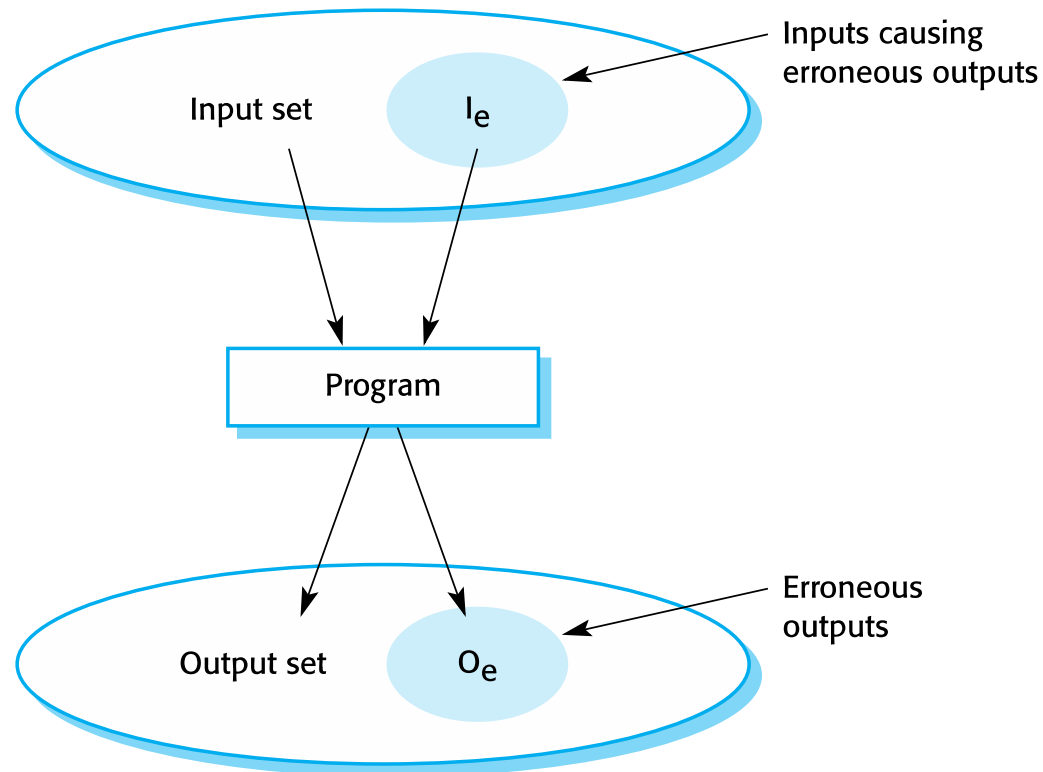
- The probability that a system, at a point in time, will be operational and able to deliver the requested services
- ✧ Both of these attributes can be **expressed quantitatively**
e.g. availability of 0.999 means that the system is up and running for 99.9% of the time.

Reliability and specifications



- ✧ **Reliability** can only be defined formally with respect to a system specification i.e. a failure is a deviation from a specification.
- ✧ Note that removing $X\%$ of the faults in a system will not necessarily improve the reliability by $X\%$.

A system as an input/output mapping



Availability perception



- ✧ Availability is usually expressed as a percentage of the time that the system is available to deliver services e.g. 99.95%.
- ✧ However, this does not take into account two factors:
 - **The number of users affected by the service outage.** Loss of service in the middle of the night is less important for many systems than loss of service during peak usage periods.
 - **The length of the outage.** The longer the outage, the more the disruption. Several short outages are less likely to be disruptive than 1 long outage. Long repair times are a particular problem.



Reliability requirements

Reliability metrics



- ✧ Reliability metrics are units of measurement of system reliability.
- ✧ System reliability is measured by counting the number of operational failures and, where appropriate, relating these to the demands made on the system and the time that the system has been operational.
- ✧ A long-term measurement programme is required to assess the reliability of critical systems.
- ✧ **Metrics**
 - Probability of failure on demand
 - Rate of occurrence of failures/Mean time to failure
 - Availability

Probability of failure on demand (POFOD)



- ✧ This is the **probability that the system will fail when a service request is made**. Useful when demands for service are intermittent and relatively infrequent.
- ✧ Appropriate for protection systems where services are demanded occasionally and where there are serious consequence if the service is not delivered.
- ✧ Relevant for many safety-critical systems with exception management components
 - Emergency shutdown system in a chemical plant.

Availability



- ✧ **Measure of the fraction of the time that the system is available for use.**
- ✧ Takes repair and restart time into account
- ✧ Availability of **0.998 means software is available for 998 out of 1000 time units.**
- ✧ Relevant for non-stop, continuously running systems
 - telephone switching systems, railway signalling systems.

Availability specification



Availability	Explanation
0.9	The system is available for 90% of the time. This means that, in a 24-hour period (1,440 minutes), the system will be unavailable for 144 minutes.
0.99	In a 24-hour period, the system is unavailable for 14.4 minutes.
0.999	The system is unavailable for 84 seconds in a 24-hour period.
0.9999	The system is unavailable for 8.4 seconds in a 24-hour period. Roughly, one minute per week.

Non-functional reliability requirements



- ✧ Non-functional reliability requirements are specifications of the required reliability and availability of a system using one of the reliability metrics (POFOD, ROCOF or AVAIL).
- ✧ Quantitative reliability and availability specification has been used for many years in safety-critical systems but is uncommon for business critical systems.
- ✧ However, as more and more companies demand 24/7 service from their systems, it makes sense for them to be precise about their reliability and availability expectations.

Benefits of reliability specification



- ✧ The process of deciding the required level of the reliability helps to clarify what stakeholders really need.
- ✧ It provides a basis for assessing when to stop testing a system. You stop when the system has reached its required reliability level.
- ✧ It is a means of assessing different design strategies intended to improve the reliability of a system.
- ✧ If a regulator has to approve a system (e.g. all systems that are critical to flight safety on an aircraft are regulated), then evidence that a required reliability target has been met is important for system certification.

Specifying reliability requirements



- ✧ Specify the availability and reliability requirements for different types of failure. There should be a lower probability of high-cost failures than failures that don't have serious consequences.
- ✧ Specify the availability and reliability requirements for different types of system service. Critical system services should have the highest reliability but you may be willing to tolerate more failures in less critical services.
- ✧ Think about whether a high level of reliability is really required. Other mechanisms can be used to provide reliable system service.

ATM reliability specification



✧ Key concerns

- To ensure that their ATMs carry out customer services as requested and that they properly record customer transactions in the account database.
- To ensure that these ATM systems are available for use when required.

✧ Database transaction mechanisms may be used to correct transaction problems so a low-level of ATM reliability is all that is required

✧ Availability, in this case, is more important than reliability

ATM availability specification



✧ System services

- The customer account database service;
 - The individual services provided by an ATM such as 'withdraw cash', 'provide account information', etc.
- ✧ The database service is critical as failure of this service means that all of the ATMs in the network are out of action.
- ✧ You should specify this to have a high level of availability.
- Database availability should be around 0.9999, between 7 am and 11pm.
 - This corresponds to a downtime of less than 1 minute per week.

ATM availability specification



- ✧ For an individual ATM, the key reliability issues depends on mechanical reliability and the fact that it can run out of cash.
- ✧ A lower level of software availability for the ATM software is acceptable.
- ✧ The overall availability of the ATM software might therefore be specified as 0.999, which means that a machine might be unavailable for between 1 and 2 minutes each day.

Insulin pump reliability specification



- ✧ Probability of failure (POFOD) is the most appropriate metric.
- ✧ Transient failures that can be repaired by user actions such as recalibration of the machine. A relatively low value of POFOD is acceptable (say 0.002) – one failure may occur in every 500 demands.
- ✧ Permanent failures require the software to be re-installed by the manufacturer. This should occur no more than once per year. POFOD for this situation should be less than 0.00002.

Functional reliability requirements



- ✧ Checking requirements that identify checks to ensure that incorrect data is detected before it leads to a failure.
- ✧ Recovery requirements that are geared to help the system recover after a failure has occurred.
- ✧ Redundancy requirements that specify redundant features of the system to be included.
- ✧ Process requirements for reliability which specify the development process to be used may also be included.

Examples of functional reliability requirements



RR1: A pre-defined range for all operator inputs shall be defined and the system shall check that all operator inputs fall within this pre-defined range. (Checking)

RR2: Copies of the patient database shall be maintained on two separate servers that are not housed in the same building. (Recovery, redundancy)

RR3: N-version programming shall be used to implement the braking control system. (Redundancy)

RR4: The system must be implemented in a safe subset of Ada and checked using static analysis. (Process)

Fault-tolerant architectures

Fault tolerance



✧ Fault tolerance:

- system can continue in operation in spite of software failure.
- Even if the system has been proved to conform to its specification, it must also be fault tolerant as
 - there may be specification errors
 - the validation may be incorrect.

Fault tolerance



✧ Fault tolerance is required:

- where there are **high availability requirements**
- where **system failure costs are very high.**

✧ In **critical situations**, software systems must be fault tolerant.

Fault-tolerant system architectures



✧ Fault-tolerant architectures:

- Generally, all based on **redundancy** and **diversity**.

✧ Examples of situations where dependable architectures are used:

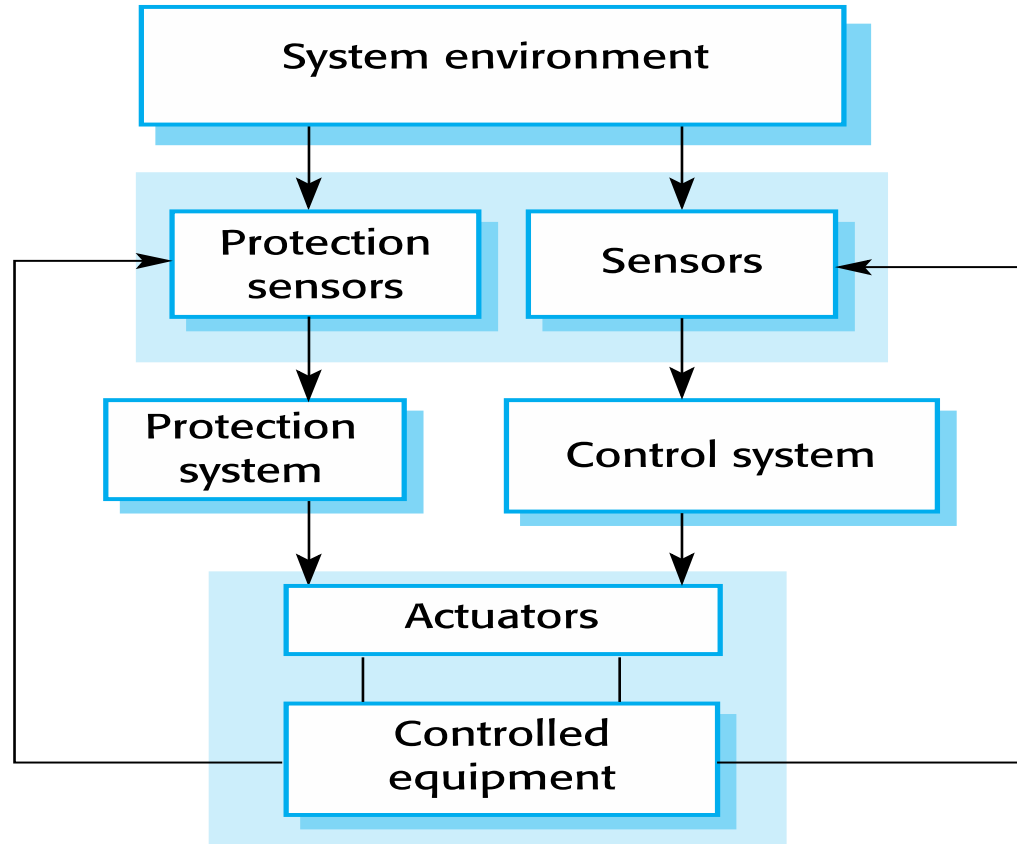
- **Flight control systems**, where system failure could threaten the safety of passengers
- **Reactor systems** where failure of a control system could lead to a chemical or nuclear emergency
- **Telecommunication systems**, where there is a need for 24/7 availability.

Protection systems



- ✧ **Protection systems** - independently monitor the controlled system and the environment.
- ✧ **If a problem is detected, it issues commands to**
 - take emergency action
 - shut down the system and avoid a catastrophe.
- ✧ **Example**
 - System to stop a train if it passes a red light
 - System to shut down a reactor if temperature/pressure are too high

Protection system architecture



Protection system functionality



✧ Protection systems

- **Redundant** - include monitoring and control capabilities.
- **Diverse** and use different technology from the control software.

✧ **Aim** is to ensure that there is a low probability of failure on demand for the protection system.

Self-monitoring architectures



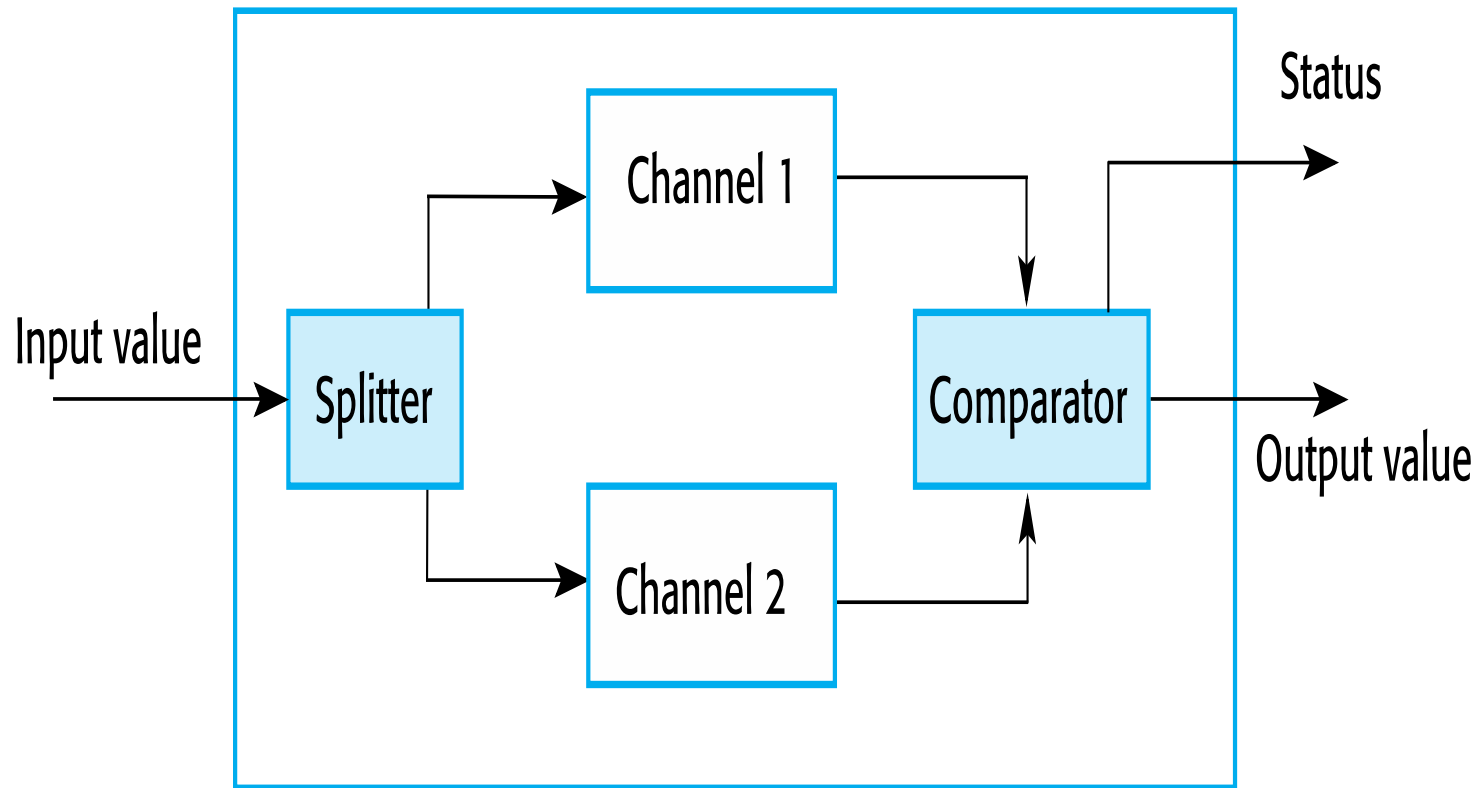
✧ Self-monitoring architectures:

- **Multi-channel architectures** where the system monitors its own operations and takes action

✧ How it works:

- The **same computation is carried out on each channel** and the results are compared.
- **If the results are identical** and are produced at the same time, then it is assumed that the system is operating correctly.
- **If the results are different**, then a failure is assumed, and a **failure exception** is raised.

Self-monitoring architecture



Self-monitoring systems



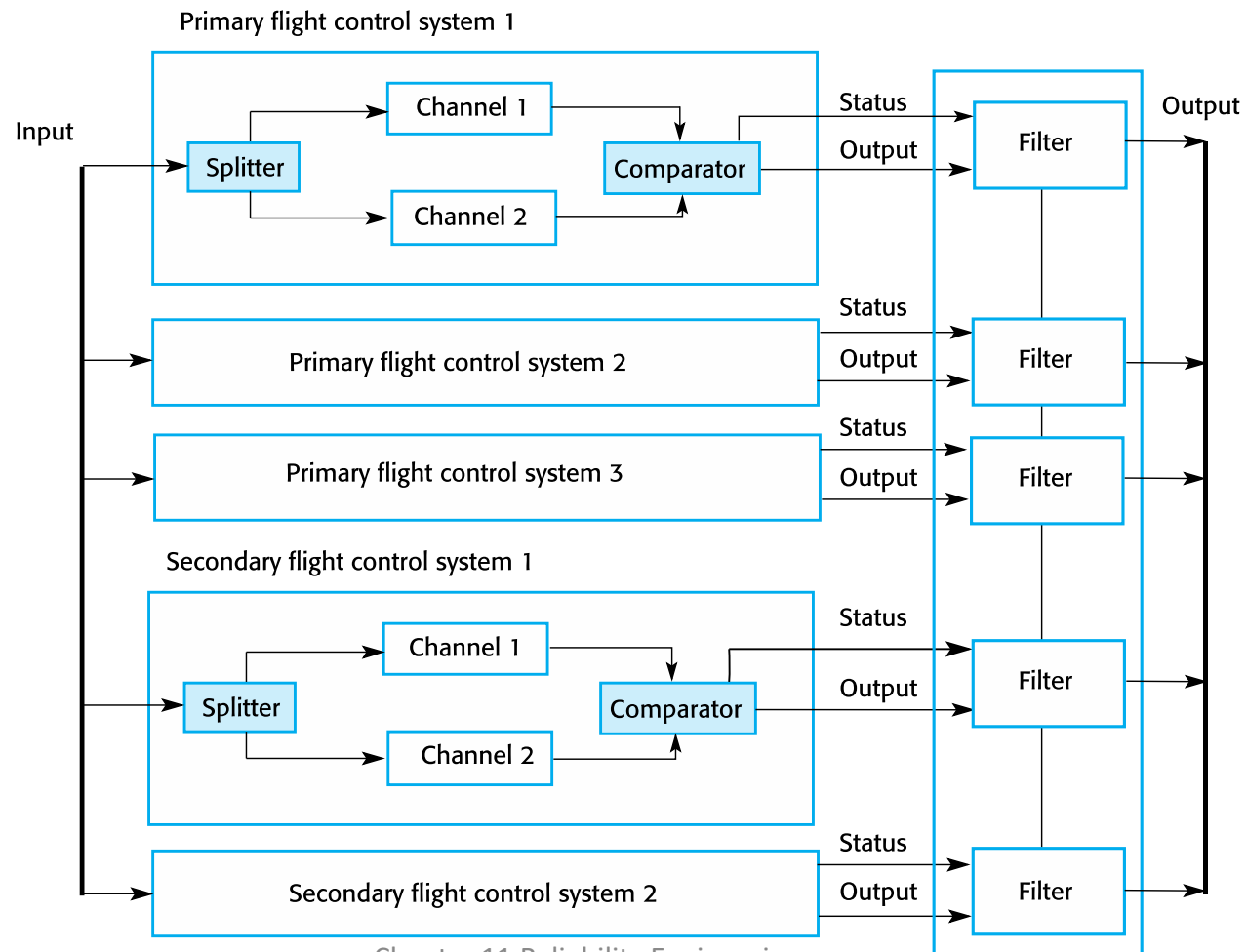
✧ If **high-availability** is required:

- use several **self-checking systems in parallel**.
- **Approach is used in the Airbus family of aircraft for their flight control systems.**

✧ Airbus flight control system architecture (Study)

Airbus flight control system architecture

Input value



Airbus architecture discussion



- ✧ The Airbus FCS has **5 separate computers**, any one of which can run the control software.
- ✧ Extensive use has been made of **diversity**
 - **Primary systems** use a **different processor** from the secondary systems.
 - **Primary and secondary systems** use **chipsets from different manufacturers**.
 - **Software in secondary systems** is **less complex** than in primary system – provides only critical functionality.
 - **Software in each channel** is developed in **different programming languages by different teams**.
 - **Different programming languages used** in primary and secondary systems.

N-version programming



✧ N-version programming:

- The **different system versions are designed and implemented by different teams.**
- It is assumed that there is **a low probability that they will make the same mistakes.**
- There is some **empirical evidence that teams commonly misinterpret specifications in the same way** and chose the same algorithms in their systems.

N-version programming

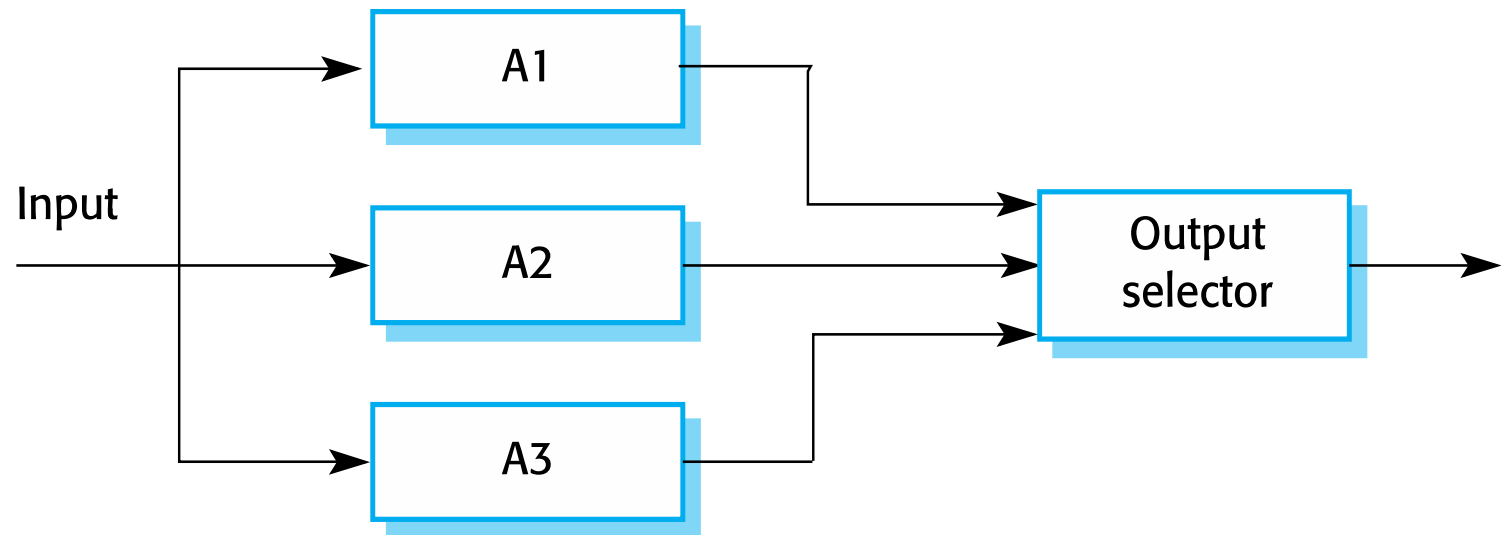


✧ Multiple versions of a software system

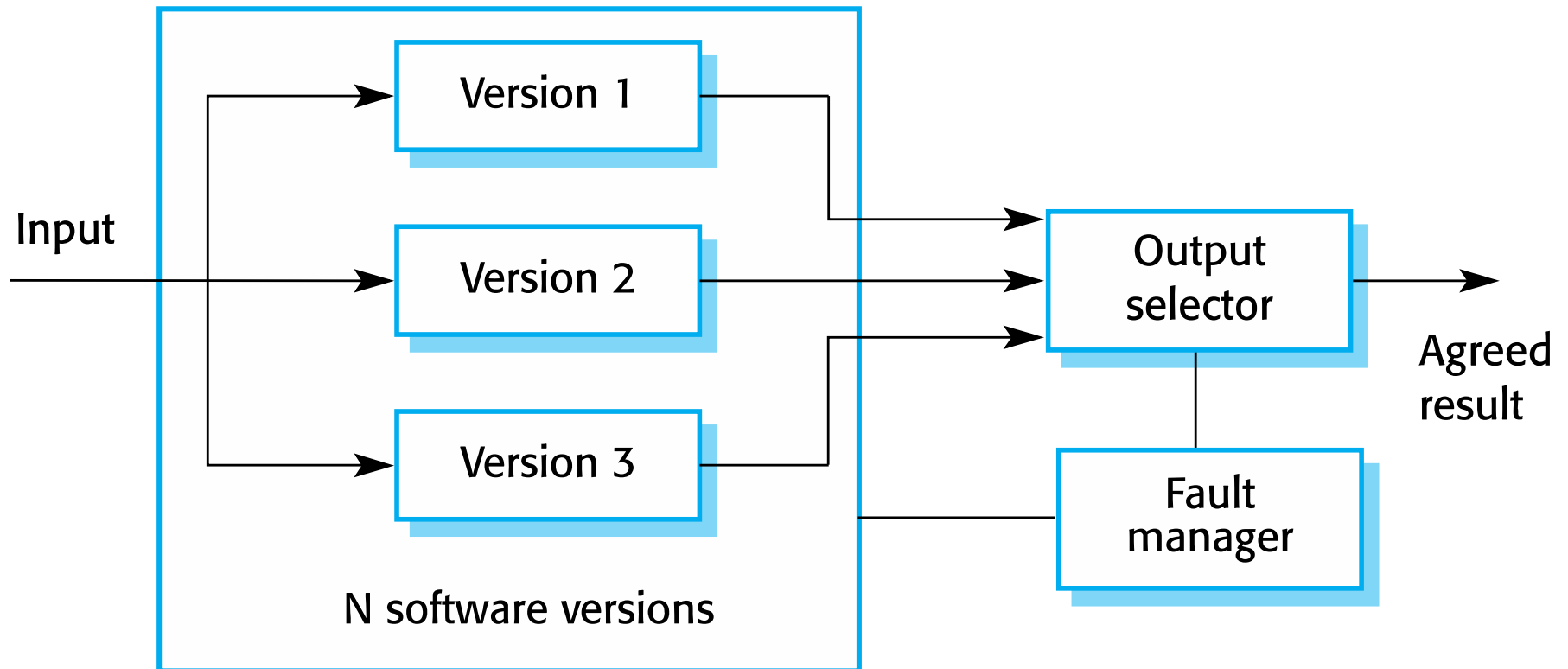
- carry out **computations at the same time** - odd number of computers involved, typically 3.
- The **results are compared using a voting system** - majority result is taken to be the correct result.

✧ Triple-modular redundancy (TMR) - Approach derived from the notion.

Triple modular redundancy



N-version programming





✧ Approaches to software fault tolerance

- **depend on software diversity** where it is assumed that different implementations of the same software specification will fail in different ways.
- It is assumed that **implementations are**
 - (a) independent
 - (b) do not include common errors.

✧ Strategies to achieve diversity

- Different programming languages
- Different design methods and tools
- Explicit specification of different algorithms

Problems with design diversity



✧ Teams are not culturally diverse, so they tend to tackle problems in the same way.

✧ Characteristic errors

- Different teams make the same mistakes.
- Specification errors - if there is an error in the specification then this is reflected in all implementations



Programming for reliability

Dependable programming



✧ **To reduce the incidence of program faults.**

- Good programming practices

✧ **The Programming practices support**

- Fault avoidance
- Fault detection
- Fault tolerance

Good practice guidelines for dependable programming



Dependable programming guidelines

1. **Limit the visibility of information in a program**
2. **Check all inputs for validity**
3. **Provide a handler for all exceptions**
4. **Minimize the use of error-prone constructs**
5. **Provide restart capabilities**
6. **Check array bounds**
7. **Include timeouts when calling external components**
8. **Name all constants that represent real-world values**

(1) Limit the visibility of information in a program



- ✧ **Program components** should only be **allowed access to data that they need for their implementation**.
 - This means that **accidental corruption of parts of the program state by these components is impossible**.

(2) Check all inputs for validity



- ✧ **All program take inputs from their environment and make assumptions** about these inputs.
 - This will help **inputs to be consistent with these assumptions**.
 - **Always check inputs before processing** against the assumptions made about these inputs.

Validity checks



✧ Range checks

- Check that the **input falls within a known range**.

✧ Size checks

- Check that the **input does not exceed some maximum size** e.g. 40 characters for a name.

✧ Representation checks

- Check that the **input does not include characters that should not be part of its representation** e.g. names do not include numerals.

✧ Reasonable checks

- Use information about the **input to check if it is reasonable** rather than an extreme value.

(3) Provide a handler for all exceptions



✧ A program exception

- an **error or some unexpected event** such as a **power failure**.

✧ Exception handling constructs

- a **mechanism to provide some fault tolerance** – to detect exceptions.

✧ Using normal control constructs

- to detect exceptions needs **many additional statements to be added to the program**.

(4) Minimize the use of error-prone constructs



- ✧ **Program faults** are usually a consequence of human error
 - because programmers lose track of the relationships between the different parts of the system

- ✧ Avoid or minimize the use of error-prone constructs!

Error-prone constructs



- ✧ **Unconditional branch (goto) statements**
- ✧ **Floating-point numbers**
 - The **imprecision** may lead to invalid comparisons.
- ✧ **Pointers referring to the wrong memory areas**
- ✧ **Dynamic memory allocation**
 - **Run-time allocation** can cause memory overflow.

Error-prone constructs



✧ Parallelism

- Can result in subtle timing errors because of **unforeseen interaction between parallel processes**.

✧ Recursion

- Errors in recursion **can cause memory overflow as the program stack fills up**.

✧ Interrupts

- Interrupts **can cause a critical operation to be terminated** and make a program difficult to understand.

✧ Inheritance

- Code is not localised. This **can result in unexpected behaviour when changes are made**

(5) Provide restart capabilities



- ✧ **For systems that involve long transactions or user interactions:**
 - provide a restart capability that allows the system to restart after failure without users having to redo everything that they have done.
- ✧ **Restart depends on the type of system**
 - Keep copies of forms so that users don't have to fill them in again if there is a problem
 - Save state periodically and restart from the saved state

(6) Check array bounds



✧ Always check that an array access is within the bounds of the array:

- Programming languages, such as C - it is possible to address a memory location outside of the range allowed for in an array declaration.
- This may lead to the well-known '**bounded buffer**' vulnerability
 - where attackers may deliberately write executable code into memory.

(7) Include timeouts when calling external components



✧ Always include timeouts on all calls to external components.

- After a defined time period has elapsed without a response, your system should then assume failure and take whatever actions are required to recover from this.

(8) Name all constants that represent real-world values



- ✧ Always give constants that reflect real-world values (such as tax rates) names rather than using their numeric values
 - always refer to them by name
 - when these 'constants' change (for sure, they are not really constant), then you only have to make the change in one place in your program.

Reliability measurement

Reliability measurement



✧ Reliability metrics include

- probability of failure on demand (POFOD),
- rate of occurrence of failure (ROCOF) and
- availability (AVAIL).

✧ To assess the reliability of a system, collect data about its operation. The data required may include:

- The number of system failures given a number of requests for system services.
- This is used to measure the **POFOD**. This applies irrespective of the time over which the demands are made.

Reliability measurement



- ✧ The time or the **number of transactions between system failures plus the total elapsed time** or total number of transactions.
 - This is used to measure **ROCOF** and **MTTF**(Mean Time To Failure)
- ✧ **MTTF** - reliability metric that measures the **average time expected until the first failure of a piece of equipment** or system under normal operating conditions
- ✧ This is used in the **measurement of availability**.
 - **Availability** does not just depend on the time between failures but **also on the time required to get the system back into operation**.

Reliability testing



✧ Reliability testing (Statistical testing)

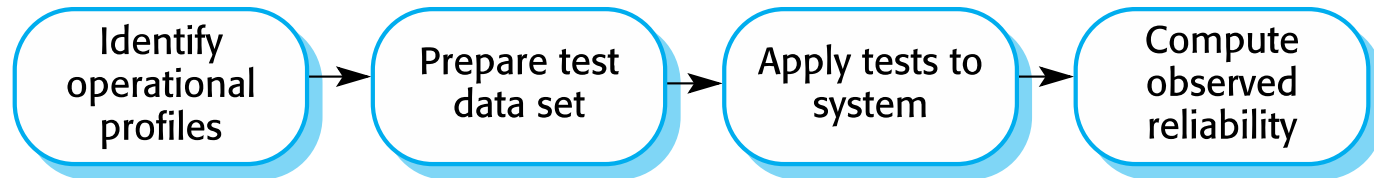
- involves running the program to assess whether or not it has reached the required level of reliability.

Statistical testing



- ✧ Testing software for reliability rather than fault detection.
- ✧ Measuring the number of errors allows the reliability of the software to be predicted.
- ✧ An acceptable level of reliability should be specified and the software tested and amended until that level of reliability is reached.

Reliability measurement



Reliability measurement problems



✧ Operational profile uncertainty

- The **operational profile may not be an accurate** reflection of the real use of the system.

✧ High costs of test data generation

- **Costs can be very high** if the test data for the system cannot be generated automatically.

✧ Statistical uncertainty

- **A statistically significant number of failures to compute the reliability is needed** but highly reliable systems will rarely fail.

✧ Recognizing failure

- It is not always obvious when a failure has occurred as there may be conflicting interpretations of a specification.

Key points



- ✧ Software reliability can be achieved by avoiding the introduction of faults, by detecting and removing faults before system deployment and by including fault tolerance facilities that allow the system to remain operational after a fault has caused a system failure.
- ✧ Reliability requirements can be defined quantitatively in the system requirements specification.
- ✧ Reliability metrics include probability of failure on demand (POFOD), rate of occurrence of failure (ROCOF) and availability (AVAIL).

Key points



- ✧ **Functional reliability requirements** are requirements for system functionality, such as checking and redundancy requirements, which help the system meet its non-functional reliability requirements.
- ✧ **Dependable system architectures** are system architectures that are designed for fault tolerance.
- ✧ There are a **number of architectural styles that support fault tolerance** including protection systems, self-monitoring architectures and N-version programming.

Key points



- ✧ **Software diversity** is difficult to achieve because it is practically impossible to ensure that each version of the software is truly independent.
- ✧ **Dependable programming** relies on including redundancy in a program as checks on the validity of inputs and the values of program variables.
- ✧ **Statistical testing** is used to estimate software reliability. It relies on testing the system with test data that matches an operational profile, which reflects the distribution of inputs to the software when it is in use.